ThesisTrack

Code Guide

How the system works, file by file, for review and learning

Covers the server, the database, the routes of each role, the pages, notifications, security and the AI modules.

Prepared for the thesis group's review. October 2026.

Table of Contents

1. About This Guide 5
2. The Big Picture 6
2.1 What kind of program is ThesisTrack? 6
2.2 Technologies and what each one does 6
2.3 The folder map 6
2.4 How one request travels through the server 7
3. Starting the Server: server.js 9
3.1 Settings and the PROD switch 9
3.2 Security headers 9
3.3 Sessions: remembering who you are 10
3.4 The per-request setup 10
3.5 CSRF protection 11
3.6 Mounting the routes 12
3.7 Errors and starting up 12
4. The Database: db.js 13
4.1 Connecting 13
4.2 The tables 13
4.3 Reading a table definition 16
4.4 Creating and updating tables safely 16
4.5 The three query helpers 17
4.6 Transactions: all or nothing 17
5. Shared Helpers: lib.js 18
5.1 Constants 18
5.2 Dates 18
5.3 The Delayed and Completed rules 18
5.4 Small HTML helpers 18
5.5 A query worth studying 19
5.6 Who may see a group? 19
5.7 Notifications and the activity log 19
6. Security Building Blocks 21
6.1 The three checks in middleware.js 21
6.2 Passwords are hashed 21
6.3 Protecting the sign-in 22
6.4 Sessions stored in MySQL 23

6.5 Safe file uploads 23
6.6 One-time codes and reset links 24
6.7 Never trust the address 24
6.8 Spreadsheet formula injection 25
6.9 Summary 25
7. Walking Through the Routes 26
7.1 How a route file is shaped 26
7.2 routes/auth.js: accounts 26
7.3 routes/student.js: the student's world 26
7.4 routes/adviser.js: the adviser's world 28
7.5 routes/admin.js: the coordinator 29
7.6 routes/shared.js: pages everyone uses 30
8. Pages and Styling: views and public 31
8.1 EJS in five minutes 31
8.2 The shared layout 31
8.3 A page that groups data: notifications.ejs 31
8.4 CSS 32
8.5 Small scripts in the browser 32
9. Notifications, Reminders and Email 33
9.1 Notifications 33
9.2 Deadline reminders: reminders.js 33
9.3 Email: mailer.js 34
9.4 First-run data: seed.js and bootstrap.js 34
10. The AI Modules 35
10.1 ai.js: one door to the AI 35
10.2 textextract.js: reading files 37
10.3 plagiarism.js: how the similarity score works 37
10.4 routes/ai.js: the routes around the tools 38
11. Following a Feature from Click to Database 41
11.1 A student uploads Chapter 4 41
11.2 The adviser returns it for revision 41
11.3 A student runs the similarity check 41
12. Running, Testing and Backing Up 43
12.1 The npm commands 43
12.2 The smoke test 43
12.3 The settings (.env) 43

12.4 When something goes wrong 44
13. Where to Change Things 45
Practice exercises 45
14. Glossary 46
15. Review Questions 47

1. About This Guide

This guide explains the code of ThesisTrack so that every group member can review it before the defense and learn how a real web system is built. It describes the code as it is today, including the AI tools, the grouped notifications and the phone layout.

Who it is for

You do not need to be an expert. If you have seen basic JavaScript (variables, functions, if statements, loops), you can follow it. New ideas are explained the first time they appear, and Section 14 is a glossary of the terms used.

How to use it

- Open the project folder in VS Code next to this guide. Every code box is **copied straight from the file named above it**, so you can find it and read the lines around it.
- Read the sections in order the first time. Later, use Section 13 ("Where to change things") when you need to change something, and Section 15 to test yourself.
- Inline names such as `server.js` or `db.get` are real file or function names. Words in bold are the key terms to remember.

> **Key idea** ThesisTrack is a **server-side rendered** web application. The browser asks for a page, the Node.js server runs JavaScript, reads the MySQL database and sends back finished HTML. There is no separate front-end framework to learn.

A good reading order

Step	What to read	Time
1	Section 2: the big picture and the folder map	20 minutes
2	Sections 3 and 4: how the server starts and how the database works	40 minutes
3	Sections 5 and 6: shared helpers and the security building blocks	40 minutes
4	Section 7: the routes of each role, with Section 11 beside it	60 minutes
5	Sections 8 to 10: pages, notifications and the AI modules	60 minutes
6	Sections 12 to 15: running, changing and testing yourself	30 minutes

2. The Big Picture

2.1 What kind of program is ThesisTrack?

ThesisTrack is a **web application**. Two programs talk to each other: the **browser** (Chrome, Edge, a phone) and the **server** (the Node.js program started by `server.js`). The browser sends an **HTTP request** such as "show me /student" or "here is a file to upload". The server decides what to do, reads or writes the **MySQL database**, and answers with a web page.

The pages are built on the server from **EJS templates** (files ending in `.ejs` in the `views` folder). A template is HTML with small holes in it where data is filled in, for example the group name or the list of chapters.

2.2 Technologies and what each one does

Technology	What it does in ThesisTrack
Node.js	Runs JavaScript on the server. Version 20 or newer is required (see `package.json`).
Express	The web framework. It receives requests, picks the right function (a **route**) and sends the response.
EJS	The template language that turns data into HTML pages.
MySQL (XAMPP)	The database that stores users, groups, chapters, comments, notifications and everything else.
mysql2	The library that lets Node.js send SQL to MySQL. The project uses its promise version, so code can use `await`.
express-session	Remembers who is signed in between requests. The sessions are saved in MySQL by `sessionstore.js`.
bcryptjs	Turns passwords into one-way hashes so real passwords are never stored.
multer	Reads uploaded files (chapters and ID photos) and saves them in the `uploads` folder.
nodemailer	Sends email (invitation codes and password reset links) through Gmail, or writes to `outbox/mail.log` when email is not set up.
pdf-parse and mammoth	Read the text out of PDF and DOCX files for the plagiarism check and grammar tool.
fetch (built into Node)	Calls the free AI service (Google Gemini, or Ollama on the school computer).
Plain CSS	All styling is in `public/style.css`. There is no CSS framework.

2.3 The folder map

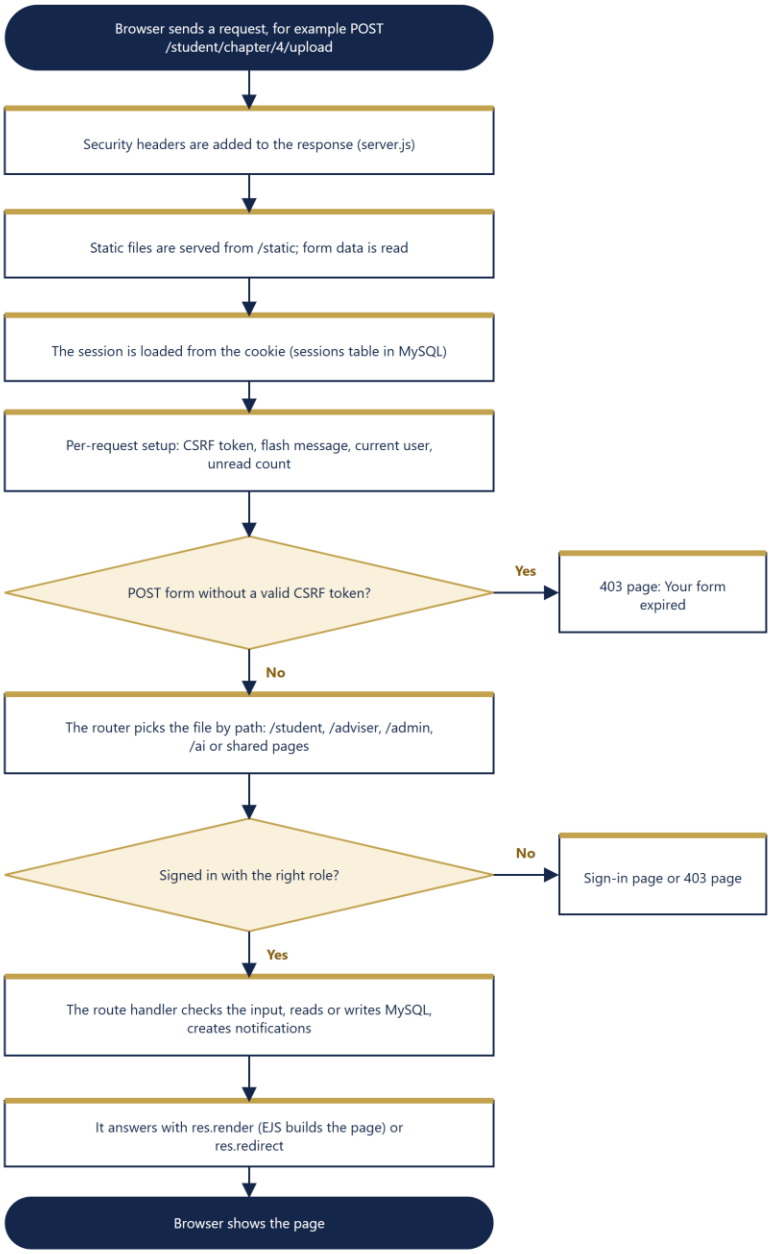
File or folder	What it is for
server.js	The starting point. Sets up security, sessions, routes and starts listening for requests.

File or folder	What it is for
db.js	Connects to MySQL, creates all tables and offers the helpers `all`, `get`, `run` and `transaction`.
lib.js	Shared helpers: chapter names, status rules, date formatting, common queries, `notify` and `log`.
middleware.js	The checks that run before a route: signed in, right role, valid CSRF token.
routes/auth.js	Sign in, sign out, student registration, adviser registration, forgot and reset password.
routes/student.js	Everything a student does: group, chapters, uploads, consultations.
routes/adviser.js	Everything an adviser does: accept groups, review chapters, deadlines, consultations.
routes/admin.js	The coordinator pages: users, invitations, panel members, groups, announcements, reports, activity log.
routes/shared.js	Pages every role uses: file downloads, notifications, My account.
routes/ai.js	The AI check page (plagiarism and grammar) and the chatbot endpoint.
ai.js, plagiarism.js, textextract.js	The AI modules: talking to Gemini or Ollama, comparing chapters, reading PDF and DOCX text.
reminders.js	Sends deadline reminders once an hour.
mailer.js	Sends email, or saves it to a log file when email is not configured.
sessionstore.js	Stores login sessions in MySQL.
bootstrap.js and seed.js	Create the first coordinator on a live site, or sample data on your own computer.
loadenv.js	Reads the private `.env` settings file.
views/	The EJS pages. `views/partials` holds the shared header, footer and chat widget.
public/	Files the browser downloads directly: `style.css` and fonts.
uploads/	Uploaded chapters and ID photos, saved under random names.
scripts/	`smoke-test.js` (automatic checks) and `backup.js` (database and file backup).
docs/	The Word documents, flowcharts and the defense guide.

2.4 How one request travels through the server

Figure 1 follows a single request, for example a student uploading a chapter. Every request in the whole system takes this same path, and each box matches a part of `server.js` that Section 3 explains.

Figure 1. How one request travels through the server



The order of the boxes is important. In Express, **middleware** (code that runs before the route) runs in the order it is written in `server.js`. For example, the session must be loaded before the code that asks "who is the current user?", and the CSRF check needs the session too.

3. Starting the Server: server.js

`server.js` is the first file Node runs (`npm start` runs `node server.js`). It is short because the real work is in other files. This section follows it from top to bottom.

3.1 Settings and the PROD switch

The first line, `require("./loadenv")`, reads the `.env` file so that secrets (email password, AI key, session secret) live outside the code and are never committed to git. Settings are called **environment variables** and are read with `process.env.NAME`.

loadenv.js: reading the .env file

```
const file = path.join(__dirname, '.env');
if (fs.existsSync(file) && typeof process.loadEnvFile === 'function') {
  try {
    process.loadEnvFile(file);
  } catch (e) {
    console.warn(`Could not read .env: ${e.message}`);
  }
}
```

server.js: PROD means the live site

```
const PROD = process.env.NODE_ENV === 'production';
const SESSION_MS = 1000 * 60 * 60 * 8;
if (PROD && !process.env.SESSION_SECRET) {
  console.error('Set SESSION_SECRET before running in production.');
  process.exit(1);
}
```

When `NODE_ENV=production`, the site is "live": cookies are HTTPS-only, there are no sample accounts and a `SESSION_SECRET` is required, otherwise the server refuses to start.

3.2 Security headers

Every response gets extra headers that tell the browser to be careful.

server.js: security headers

```
app.use((req, res, next) => {
  res.set({
    'X-Content-Type-Options': 'nosniff',
    'X-Frame-Options': 'DENY',
    'Referrer-Policy': 'same-origin',
    // The pages use small inline scripts, so script-src is not restricted; these directives
still block framing, plugins and form hijacking.
    'Content-Security-Policy': "frame-ancestors 'none'; base-uri 'self'; form-action 'self';
object-src 'none'",
    'Permissions-Policy': 'camera=(self), microphone=(), geolocation=()',
    'Cross-Origin-Opener-Policy': 'same-origin',
  });
  if (PROD) res.set('Strict-Transport-Security', 'max-age=15552000; includeSubDomains');
  next();
});
```

Header	What it stops
X-Content-Type-Options: nosniff	The browser guessing a file type and running an upload as a script.

Header	What it stops
X-Frame-Options: DENY and frame-ancestors	Another website showing ThesisTrack inside a hidden frame (clickjacking).
Referrer-Policy: same-origin	Leaking page addresses to other sites.
Content-Security-Policy	Pages being framed or forms posting to another website.
Permissions-Policy: camera=(self)	Only ThesisTrack itself may use the camera (for the ID photo).
Strict-Transport-Security (live site only)	Browsers using plain http after the first https visit.

3.3 Sessions: remembering who you are

HTTP forgets everything between requests. A **session** fixes that: after login the server gives the browser a random cookie, and keeps the real data (the user id) on the server under that cookie.

server.js: session setup

```
app.use(
  session({
    secret: process.env.SESSION_SECRET || 'dev-only-secret-change-me',
    resave: false,
    saveUninitialized: false,
    store: new MySQLStore(SESSION_MS),
    // No maxAge: a browser-session cookie, so closing the browser signs the user out. SESSION_MS
still caps the server-side lifetime.
    cookie: { httpOnly: true, sameSite: 'lax', secure: PROD },
  })
);
```

- `secret` signs the cookie so it cannot be forged.
- `store: new MySQLStore(...)` saves sessions in the `sessions` table, so people stay signed in when the server restarts.
- `httpOnly` means page scripts cannot read the cookie; `sameSite: lax` limits cross-site sending; `secure: PROD` means HTTPS-only on the live site.
- There is no `maxAge`, so it is a browser-session cookie: closing the browser signs the user out.

3.4 The per-request setup

This middleware runs for every request. It prepares things the pages need.

server.js: runs on every request

```
// Per-request setup: CSRF token, flash message, current user, helpers for the views.
app.use(async (req, res, next) => {
  if (!req.session.csrf) req.session.csrf = crypto.randomBytes(24).toString('hex');
  req.flash = (type, msg) => { req.session.flash = { type, msg }; };
  res.locals.flash = req.session.flash || null;
  delete req.session.flash;
  res.locals.csrf = req.session.csrf;
  res.locals.L = L;
  res.locals.showDemo = !PROD;
  res.locals.path = req.path;
  res.locals.user = null;
  res.locals.unread = 0;
  if (req.session.userId) {
    const u = await db.get('SELECT id, name, email, role, active FROM users WHERE id = ?',
[req.session.userId]);
    if (u && u.active) {
      req.user = u;
      res.locals.user = u;
      res.locals.unread = (await db.get('SELECT COUNT(*) AS n FROM notifications WHERE user_id =
? AND is_read = 0', [u.id])).n;
    } else {
      delete req.session.userId;
    }
  }
  next();
});
```

- **CSRF token**: a random value kept in the session and put in every form (Section 3.5).
- **Flash message**: `req.flash` saves a one-time message in the session. The next page shows it, then `delete req.session.flash` removes it. That is how "Group created." appears once after a redirect.
- **res.locals**: values placed here (the user, the unread count, the helper library `L`) are available in every EJS page without being passed each time.
- The current user is loaded from the database each time, so a deactivated account is signed out immediately.

3.5 CSRF protection

CSRF (cross-site request forgery) is an attack where another website makes your browser send a hidden form to ThesisTrack while you are signed in. The defense is a secret token in each form that the attacker cannot know.

server.js: every POST must carry the token

```
// Forms that are not file uploads are checked here. Upload routes check after multer has read
the body.
app.use((req, res, next) => {
  if (req.method === 'POST' && !(req.headers['content-type'] || '').startsWith('multipart/'))
return csrfCheck(req, res, next);
  next();
});
```

Forms contain `<input type="hidden" name="_csrf" value="<%= csrf %>">`. `csrfCheck` (in `middleware.js`) compares it with the one in the session. File upload forms are checked later, inside their own route, because the form body can only be read after multer has processed the upload.

3.6 Mounting the routes

server.js: which file handles which address

```
app.use('/', require('./routes/auth'));
app.use('/', require('./routes/shared'));
app.use('/ai', require('./routes/ai'));
app.use('/student', requireRole('student'), require('./routes/student'));
app.use('/adviser', requireRole('adviser'), require('./routes/adviser'));
app.use('/admin', requireRole('admin'), require('./routes/admin'));
```

`app.use("/student", requireRole("student"), ...)` means: any address starting with `/student` first passes `requireRole("student")` and only then reaches `routes/student.js`. This is why a student who types `/admin` in the address bar gets a "No access" page.

3.7 Errors and starting up

After the routes come two catch-all handlers: one for pages that do not exist (404) and one for errors. In Express 5, an error thrown inside an `async` route is passed to the error handler automatically, which then shows a friendly page. If the database is down it says so and reminds you to start MySQL in XAMPP.

server.js: booting the system

```
(async () => {
  try {
    await db.init();
    if (PROD) await ensureAdmin();
    else if (await seedIfEmpty()) console.log('First run: sample data created. Every sample
account uses the password "password123".');
  } catch (e) {
    console.error(e.message);
    process.exit(1);
  }
  startReminders();
  app.listen(PORT, () => console.log(`ThesisTrack is running at http://localhost:${PORT} (MySQL
database "${db.DB_NAME}")`));
})();
```

1. `db.init()` connects to MySQL and creates the database and tables if they are missing.
2. On a live site, `ensureAdmin()` creates the first coordinator from `ADMIN_EMAIL` and `ADMIN_PASSWORD`. On your own computer, `seedIfEmpty()` fills an empty database with sample accounts.
3. `startReminders()` starts the hourly deadline reminders.
4. `app.listen` opens the port (3000 by default) and the system is ready.

> **Try it** Start the server, then open http://localhost:3000/healthz. It answers "ok" when the database is reachable. Stop MySQL in XAMPP and refresh to see "database unavailable".

4. The Database: db.js

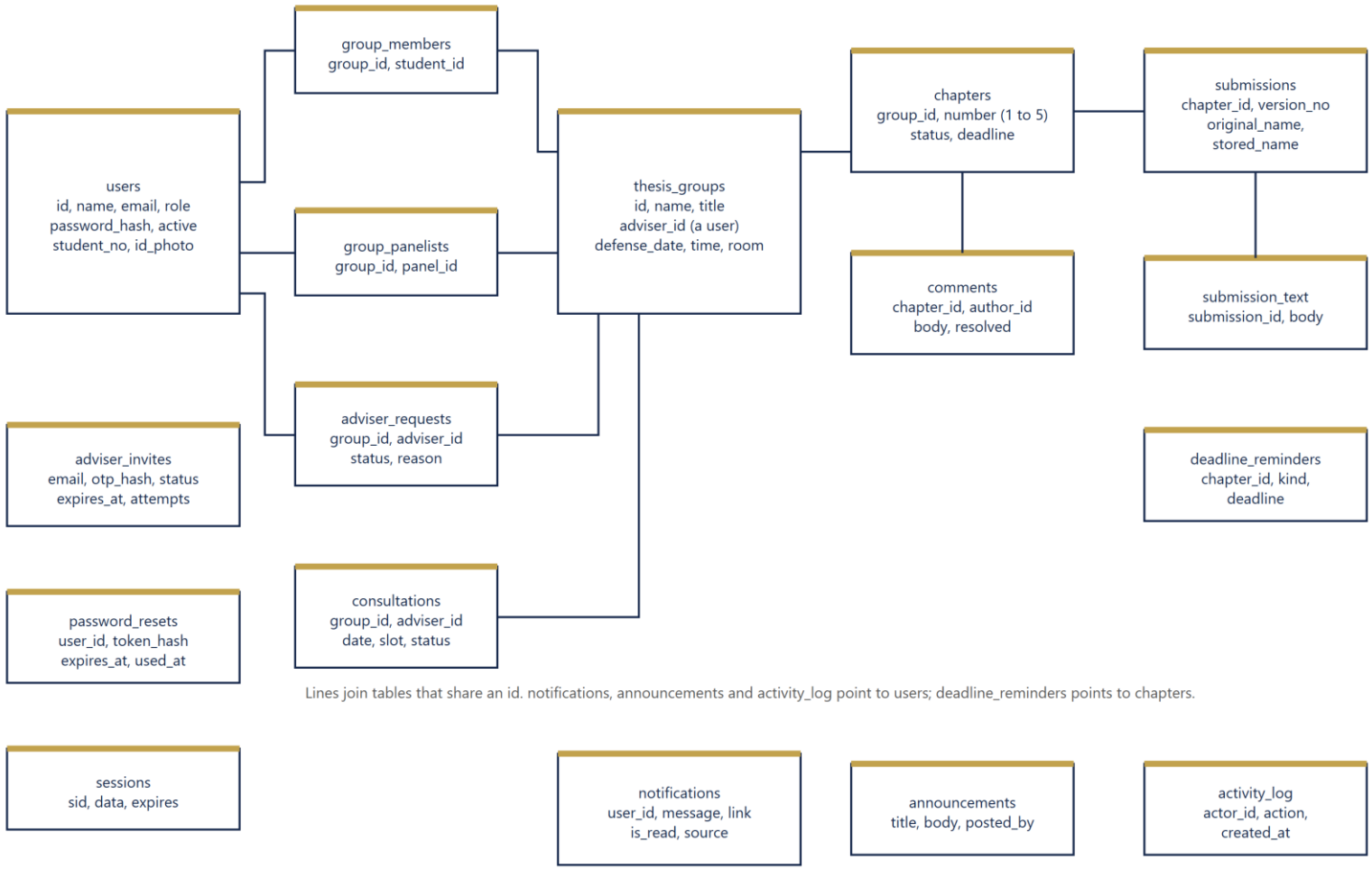
4.1 Connecting

`db.js` holds the connection settings. The defaults match a fresh XAMPP install (host localhost, user root, no password, database `thesistrack`), and every value can be changed with an environment variable. It uses a **connection pool**: a small set of ready connections (10) that requests borrow and return, which is faster than opening a new connection each time.

4.2 The tables

Figure 2 (next page) shows how the tables connect. Each table stores one kind of thing. Lines mean that one table keeps the **id** of a row in another table. That stored id is called a **foreign key**.

Figure 2. How the database tables connect



Lines join tables that share an id. notifications, announcements and activity_log point to users; deadline_reminders points to chapters.

Table	What it stores
users	Every account: students, advisers, the coordinator, and panel members (records only). The `role` column says which. Passwords are stored as hashes.
thesis_groups	One row per group: name, title, adviser, defense date, time and room.
group_members	Which student belongs to which group. `student_id` is unique, so a student can be in only one group.
group_panelists	Which panel members are attached to a group's defense.
adviser_requests	The coordinator's request to an adviser (Pending, Accepted, Declined or Cancelled) and the decline reason.
adviser_invites	Adviser invitations with a hashed one-time code, its expiry and the number of wrong tries.
chapters	Five rows per group. Holds the status and deadline of each chapter.
submissions	Every uploaded version of a chapter: version number, original file name and the random stored name.
submission_text	The text read from a submission, saved once so the AI checks do not read the file again.
comments	Adviser feedback on a chapter and whether the students marked it resolved.
consultations	Meeting requests: date, time slot, topic and status.
notifications	Messages for users, with a link, a read flag and who sent it (`source`).
announcements	Coordinator announcements.
activity_log	A record of important actions: who did what and when.
password_resets	Hashed one-time password reset links with expiry.
deadline_reminders	Remembers which reminder was already sent so nobody gets the same one twice.
sessions	Login sessions.

4.3 Reading a table definition

db.js: chapters and submissions

```
  `CREATE TABLE IF NOT EXISTS chapters (
    id         INT AUTO_INCREMENT PRIMARY KEY,
    group_id   INT NOT NULL,
    number     INT NOT NULL,
    status     ENUM('none','submitted','revision','approved','rejected') NOT NULL DEFAULT 'none',
    deadline   DATE NULL,
    UNIQUE (group_id, number),
    FOREIGN KEY (group_id) REFERENCES thesis_groups(id) ON DELETE CASCADE
  )`,
  `CREATE TABLE IF NOT EXISTS submissions (
    id             INT AUTO_INCREMENT PRIMARY KEY,
    chapter_id     INT NOT NULL,
    version_no     INT NOT NULL,
    original_name  TEXT NOT NULL,
    stored_name    VARCHAR(100) NOT NULL,
    uploaded_by    INT NULL,
    uploaded_at    DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    UNIQUE (chapter_id, version_no),
    FOREIGN KEY (chapter_id) REFERENCES chapters(id) ON DELETE CASCADE,
    FOREIGN KEY (uploaded_by) REFERENCES users(id)
  )`,
```

- `AUTO_INCREMENT PRIMARY KEY`: MySQL gives each new row the next id.
- `ENUM(...)`: the column can only hold one of the listed values, so a chapter can never have a made-up status.
- `UNIQUE (group_id, number)`: a group cannot have two Chapter 3 rows. `UNIQUE (chapter_id, version_no)` stops two Version 2 rows.
- `FOREIGN KEY ... ON DELETE CASCADE`: if a group is deleted, its chapters, and then their submissions, are deleted too. The database keeps itself tidy.

4.4 Creating and updating tables safely

Every definition starts with `CREATE TABLE IF NOT EXISTS`, so starting the server again never destroys data. When a column is added later (for example `student_no`, `consent_at` or the notification `source`), a database made before that change still needs it. `addColumnIfMissing` checks and adds it:

db.js: a tiny migration helper

```
async function addColumnIfMissing(table, column, definition) {
  const [rows] = await pool.query(
    'SELECT 1 FROM information_schema.columns WHERE table_schema = ? AND table_name = ? AND
column_name = ?',
    [DB_NAME, table, column]
  );
  if (rows.length) return false;
  await pool.query(`ALTER TABLE \`${table}\` ADD COLUMN \`${column}\` ${definition}`);
  return true;
}
```

When the `source` column was added, `backfillNotificationSource` also sorted the old notifications into groups by looking at their wording, so old messages did not all land under "Reminders".

4.5 The three query helpers

db.js: the helpers every route uses

```
const all = async (sql, params = []) => (await runner().query(sql, params))[0];
const get = async (sql, params = []) => (await all(sql, params))[0];
const run = async (sql, params = []) => {
  const [r] = await runner().query(sql, params);
  return { lastInsertRowid: r.insertId, changes: r.affectedRows };
};
```

Helper	Use
db.all(sql, params)	Returns a list of rows (use for lists such as chapters).
db.get(sql, params)	Returns the first row or `undefined` (use for one record, or an "exists?" check).
db.run(sql, params)	Runs INSERT, UPDATE or DELETE and returns `lastInsertRowid` and `changes`.

> **Watch out** Notice the question marks: `db.get("SELECT * FROM users WHERE email = ?", [email])`. The value travels separately from the SQL text, so even if someone types SQL into the email box it is treated as plain text. This stops **SQL injection**. Never build SQL by joining strings with user input.

4.6 Transactions: all or nothing

Some actions change several tables. Creating a group inserts the group, adds its members, and creates five chapters. If something fails halfway, you would be left with a broken group. A **transaction** makes the whole set succeed together or be cancelled together (**rollback**).

db.js: db.transaction

```
// Runs fn inside one transaction. Everything it awaits shares the same connection, and any error
rolls it back.
async function transaction(fn) {
  if (txStore.getStore()) return fn(); // already inside a transaction
  const conn = await pool.getConnection();
  try {
    await conn.beginTransaction();
    const result = await txStore.run(conn, fn);
    await conn.commit();
    return result;
  } catch (e) {
    await conn.rollback();
    throw e;
  } finally {
    conn.release();
  }
}
```

The clever part is `AsyncLocalStorage`. Inside `db.transaction(async () => { ... })`, every `db.run` automatically uses the same connection, so the route code stays simple: it just calls `db.run` as usual. You will see this in the create-group and review routes (Section 7).

5. Shared Helpers: lib.js

`lib.js` collects small tools used by many routes and pages. It is also passed to every page as `L`, which is why templates can write `L.fmtDate(c.deadline)`.

5.1 Constants

lib.js: the five chapters

```
const CH_NAMES = [
  'Introduction',
  'Review of Related Literature',
  'Methodology',
  'Results and Discussion',
  'Summary and Conclusion',
];
```

lib.js: chapter statuses with their label and colour

```
const STATUS = {
  none: { label: 'Not submitted', tone: 'muted' },
  submitted: { label: 'Pending review', tone: 'info' },
  revision: { label: 'For revision', tone: 'warn' },
  approved: { label: 'Approved', tone: 'good' },
  rejected: { label: 'Rejected', tone: 'bad' },
};
```

Chapter statuses are stored in the database as short codes (`none`, `submitted`, `revision`, `approved`, `rejected`). `STATUS` translates them to words people read ("Pending review") and a colour tone for the badge. Keeping this in one place means one change updates every page.

5.2 Dates

The database setting `dateStrings: true` returns dates as text like `2026-10-09`. Text in that order sorts correctly, so the code compares dates directly: `c.deadline < today()`. `fmtDate` turns them into "Oct 9, 2026" for display.

5.3 The Delayed and Completed rules

lib.js: overdue and health

```
// ---------- status logic ----------
const isOverdue = (c) =>
  !!c.deadline && c.deadline < today() && ['none', 'revision', 'rejected'].includes(c.status);
const healthOf = (chs) =>
  chs.length && chs.every((c) => c.status === 'approved') ? 'completed' : chs.some(isOverdue) ?
'delayed' : 'on track';
```

A chapter is **overdue** only if it has a deadline in the past and is still waiting on the students (`none`, `revision` or `rejected`). A chapter waiting for the adviser (`submitted`) is never overdue, because the students did their part. A group is `completed` when all chapters are approved, `delayed` when any chapter is overdue, otherwise `on track`. These are the same rules written in Section 6 of the system document.

5.4 Small HTML helpers

lib.js: a status badge

```
const pill = (k) => `<span class="pill ${STATUS[k].tone}">${STATUS[k].label}</span>`;
```

`pill`, `hpill`, `strip` and `bar` return ready-made HTML for the coloured badges, the five-block strip and the progress bar. They are printed unescaped in templates (`<%- ... %>`), which is safe here because they are built only from fixed constants, never from user text.

5.5 A query worth studying

lib.js: chapters with version count and latest file

```
const chaptersOf = (groupId) =>
  db.all(
    `SELECT c.*,
      (SELECT COUNT(*) FROM submissions s WHERE s.chapter_id = c.id) AS versions,
      (SELECT id FROM submissions s WHERE s.chapter_id = c.id ORDER BY version_no DESC LIMIT 1)
AS latest_id,
      (SELECT original_name FROM submissions s WHERE s.chapter_id = c.id ORDER BY version_no DESC
LIMIT 1) AS latest_name
     FROM chapters c WHERE c.group_id = ? ORDER BY c.number`,
    [groupId]
  );
```

For each chapter, three small **subqueries** ask: how many submissions exist, what is the id of the newest one, and what is its file name. This gives the dashboard everything in one database trip instead of many.

5.6 Who may see a group?

lib.js: one rule for access

```
async function canAccessGroup(user, groupId) {
  if (!user) return false;
  if (user.role === 'admin') return true;
  if (user.role === 'student') return !!(await db.get('SELECT 1 FROM group_members WHERE group_id
= ? AND student_id = ?', [groupId, user.id]));
  if (user.role === 'adviser') return !!(await db.get('SELECT 1 FROM thesis_groups WHERE id = ?
AND adviser_id = ?', [groupId, user.id]));
  return false;
}
```

This function is the **gatekeeper** for files and AI checks. A coordinator can see every group, a student only their own group, and an adviser only groups where they are the assigned adviser. Everything else is refused.

5.7 Notifications and the activity log

lib.js: notify and notifyGroup

```
// source is who the message is from: 'adviser', 'coordinator', 'student' or 'system'
(reminders). The notifications page groups by it.
const notify = (userId, message, link, source) =>
  db.run('INSERT INTO notifications (user_id, message, link, source) VALUES (?, ?, ?, ?)',
[userId, message, link || null, source || 'system']);
const notifyGroup = async (groupId, message, link, exceptId, source) => {
  for (const m of await membersOf(groupId)) if (m.id !== exceptId) await notify(m.id, message,
link, source);
};
```

`notify` saves one message for one person. `notifyGroup` sends it to every member of a group. The last argument, **source**, records who the message is from ("adviser", "coordinator", "student" or "system").

The Notifications page uses it to build the sections. `log` writes to the `activity_log` table, which is what the coordinator's Activity log page shows.

6. Security Building Blocks

A thesis system holds unpublished work and student IDs, so security is built in at several layers. Think of it as locks on several doors, not one big lock.

6.1 The three checks in middleware.js

middleware.js: the whole file

```
// Login, role and CSRF checks used by the routes.
const requireLogin = (req, res, next) => {
  if (!req.user) {
    if (req.method === 'GET') req.session.returnTo = req.originalUrl;
    return res.redirect('/login');
  }
  next();
};

const requireRole = (...roles) => (req, res, next) => {
  if (!req.user) {
    if (req.method === 'GET') req.session.returnTo = req.originalUrl;
    return res.redirect('/login');
  }
  if (!roles.includes(req.user.role)) {
    return res.status(403).render('error', { title: 'No access', message: 'Your account does not
have access to this page.' });
  }
  next();
};

// Every form carries a hidden _csrf token. A POST without the right token is refused.
const csrfCheck = (req, res, next) => {
  if (!req.body || req.body._csrf !== req.session.csrf) {
    return res.status(403).render('error', { title: 'Session expired', message: 'Your form
expired. Go back, refresh the page and try again.' });
  }
  next();
};

const homeFor = (user) => ({ student: '/student', adviser: '/adviser', admin: '/admin'
}[user.role]);
```

- **requireLogin**: if nobody is signed in, remember the page asked for and send them to the sign-in page. After login they return to it.
- **requireRole**: like requireLogin, but also checks the role. A wrong role gets a 403 "No access" page.
- **csrfCheck**: the form token must match the session token.

A **middleware function** always has three parameters: `req` (the request), `res` (the response) and `next`. Calling `next()` means "all good, continue to the next step". Sending a response instead stops the chain.

6.2 Passwords are hashed

The database never stores a password. It stores a **hash**: a scrambled value that cannot be turned back into the password. `bcrypt.hash(password, 10)` creates it (10 is the work factor; higher is slower and harder to attack). At login, `bcrypt.compare(typed, stored)` hashes the typed password the same way and compares. Even if someone stole the database, they would not see anyone's password.

6.3 Protecting the sign-in

routes/auth.js: blocking password guessing

```
// Simple in-memory brake on password guessing: 10 failed logins per IP per 15 minutes.
const fails = new Map();
const blocked = (ip) => {
  const f = fails.get(ip);
  if (!f) return false;
  if (Date.now() - f.since > 15 * 60 * 1000) { fails.delete(ip); return false; }
  return f.n >= 10;
};
const fail = (ip) => {
  const f = fails.get(ip) || { n: 0, since: Date.now() };
  f.n++;
  fails.set(ip, f);
};
```

routes/auth.js: the sign-in route

```
router.post('/login', async (req, res) => {
  const ip = req.ip;
  if (blocked(ip)) {
    req.flash('error', 'Too many failed attempts. Wait 15 minutes and try again.');
    return res.redirect('/login');
  }
  const email = String(req.body.email || '').trim();
  const user = await db.get('SELECT * FROM users WHERE email = ?', [email]);
  if (user && user.role === 'panel') {
    req.flash('error', 'Panel members do not sign in. The coordinator records them and assigns
them to groups.');
    return res.redirect('/login');
  }
  if (!user || !user.active || !await bcrypt.compare(String(req.body.password || ''),
user.password_hash)) {
    fail(ip);
    req.flash('error', 'Wrong email or password, or the account is inactive.');
    return res.redirect('/login');
  }
  // The role picked on the form must match the account, so a student cannot sign in as a
coordinator by mistake.
  if (req.body.role && req.body.role !== user.role) {
    req.flash('error', `This account is not a ${L.ROLE_LABEL[req.body.role]?.toLowerCase() ||
'member of that role'}. Choose the right role and try again.`);
    return res.redirect('/login');
  }
  fails.delete(ip);
  startSession(req, res, user);
});
```

1. After 10 failed attempts from one address (IP) within 15 minutes, further attempts are refused.
2. Panel members are records only, so they get a clear message and cannot sign in.
3. A wrong email and a wrong password give the **same message**, so nobody can discover which emails are registered.
4. The role chosen on the form must match the account.
5. On success, `startSession` creates a new session.

routes/auth.js: starting a session

```
function startSession(req, res, user) {
  const returnTo = req.session.returnTo;
  req.session.regenerate((err) => {
    if (err) throw err;
    req.session.userId = user.id;
    res.redirect(returnTo && returnTo.startsWith('/') ? returnTo : homeFor(user));
  });
}
```

`req.session.regenerate` gives the user a **new** session id at login. This prevents **session fixation**, where an attacker plants a known session id before the victim signs in.

6.4 Sessions stored in MySQL

`sessionstore.js` is a small class that teaches express-session to save sessions in the `sessions` table. It has four methods the library calls: `get` (read a session), `set` (save), `touch` (extend) and `destroy` (delete). It also deletes expired sessions every 15 minutes.

6.5 Safe file uploads

routes/student.js: multer settings

```
// ---------- upload settings: PDF, DOC, DOCX up to 10 MB, saved under a random name ----------
const ALLOWED = ['.pdf', '.doc', '.docx'];
const upload = multer({
  storage: multer.diskStorage({
    destination: db.UPLOAD_DIR,
    filename: (req, file, cb) => cb(null, crypto.randomUUID() +
path.extname(file.originalname).toLowerCase()),
  }),
  limits: { fileSize: 10 * 1024 * 1024, files: 1 },
  fileFilter: (req, file, cb) =>
    ALLOWED.includes(path.extname(file.originalname).toLowerCase()) ? cb(null, true) : cb(new
Error('WRONG_TYPE')),
});
const removeFile = (file) => file && fs.rmSync(file.path, { force: true });
```

routes/student.js: checking the real file type

```
// Checks the first bytes, so a renamed .exe or .html is not accepted as a PDF or Word file.
function looksLikeDocument(file) {
  const head = Buffer.alloc(8);
  const fd = fs.openSync(file.path, 'r');
  fs.readSync(fd, head, 0, 8, 0);
  fs.closeSync(fd);
  const ext = path.extname(file.originalname).toLowerCase();
  if (ext === '.pdf') return head.subarray(0, 4).toString('latin1') === '%PDF';
  if (ext === '.docx') return head[0] === 0x50 && head[1] === 0x4b && head[2] === 0x03 && head[3]
=== 0x04;
  return head.subarray(0, 4).equals(Buffer.from([0xd0, 0xcf, 0x11, 0xe0])); // .doc
}
```

- The file name is replaced by a random **UUID**, so nobody can guess a file's address or overwrite another file.
- Only `.pdf`, `.doc` and `.docx` up to 10 MB are accepted.
- The first bytes of the file are checked (**magic bytes**: a PDF starts with "%PDF", a DOCX starts with "PK"), so a renamed `.exe` is rejected.

- Files are not served from a public folder. They can only be downloaded through `/files/:id`, which first checks `canAccessGroup`.

6.6 One-time codes and reset links

routes/auth.js: forgot password

```
router.post('/forgot', async (req, res) => {
  const email = String(req.body.email || '').trim().toLowerCase();
  if (tooMany(req.ip)) return res.status(429).render('forgot', { title: 'Forgot password', sent:
false, error: 'Too many requests. Wait 15 minutes and try again.' });
  const user = await db.get("SELECT * FROM users WHERE email = ? AND active = 1 AND role !=
'panel'", [email]);
  if (user) {
    const token = crypto.randomBytes(32).toString('hex');
    await db.run(`INSERT INTO password_resets (user_id, token_hash, expires_at) VALUES (?, ?,
DATE_ADD(NOW(), INTERVAL ${RESET_MINUTES} MINUTE))`, [user.id, sha(token)]);
    const base = process.env.APP_URL || `${req.protocol}://${req.get('host')}`;
    try {
      await mailer.send({
        to: user.email,
        subject: 'Reset your ThesisTrack password',
        text: `Hello ${user.name},\n\nOpen this link to choose a new password. It works once and
expires in ${RESET_MINUTES} minutes:\n\n${base}/reset/${token}\n\nIf you did not ask for this,
ignore this email. Your password stays the same.\n`,
      });
    } catch (e) { console.error(e); }
    await L.log(user.id, 'asked for a password reset link');
  }
  // The same answer whether or not the email exists, so nobody can test which emails are
registered.
  res.render('forgot', { title: 'Forgot password', sent: true, mailReady: mailer.configured() });
});
```

- The reset token is 32 random bytes. Only its **SHA-256 hash** is saved, so someone who reads the database cannot use it.
- The link works once and expires in 30 minutes (`RESET_MINUTES`).
- The page gives the same answer whether or not the email exists.
- Adviser invitation codes work the same way: a 6-digit code, hashed with bcrypt, valid for 60 minutes, and locked after 5 wrong tries.

6.7 Never trust the address

Anyone can edit a number in the address bar. So every route that takes an id also checks that the signed-in person is allowed to touch that record. Compare these examples:

routes/adviser.js: an adviser can only open their own groups

```
const myGroup = (req, id) => db.get('SELECT * FROM thesis_groups WHERE id = ? AND adviser_id =
?', [Number(id), req.user.id]);
const chapterOf = (gid, n) => db.get('SELECT * FROM chapters WHERE group_id = ? AND number = ?',
[gid, Number(n)]);
const denied = (res) => res.status(403).render('error', { title: 'No access', message: 'This
group is not assigned to you.' });
```

The query includes `adviser_id = req.user.id`. If an adviser tries another group's id, the query finds nothing and the route answers "This group is not assigned to you." Students are protected the same way, because `routes/student.js` always works with `req.group` (the group found from the logged-in student), never an id from the address.

6.8 Spreadsheet formula injection

routes/admin.js: safe CSV export

```
// Spreadsheet programs run text that starts with = + - @ as a formula, so those cells get a
leading apostrophe.
const csvCell = (v) => {
  let s = String(v ?? '');
  if (/^[=+\-@\t\r]/.test(s)) s = "'" + s;
  return /[",\n]/.test(s) ? `"${s.replace(/"/g, '""')}"` : s;
};
```

If a cell starts with `=`, `+`, `-` or `@`, Excel may run it as a formula. The helper adds a leading apostrophe so exported reports are safe to open.

6.9 Summary

Danger	Defense	Where
Guessing passwords	Hashing, block after 10 failures, same error message	routes/auth.js
Stolen session	httpOnly cookie, new session on login, secure cookie on the live site	server.js, auth.js
Forged forms (CSRF)	Secret token in every form	server.js, middleware.js
SQL injection	Placeholders (`?`) in every query	db.js and all routes
Harmful uploads	Type, size and first-byte checks, random names	routes/student.js, auth.js
Seeing other groups' work	`canAccessGroup` and ownership checks	lib.js and routes
Script injection (XSS)	EJS escapes text by default	views
Spreadsheet formulas	Leading apostrophe	routes/admin.js

7. Walking Through the Routes

7.1 How a route file is shaped

Each file in `routes` creates a **router**, attaches handlers to it, and exports it. A handler is a function that receives the request and sends a response. Almost every handler follows the same pattern:

1. Read the input: `req.body` (form fields), `req.params` (parts of the address such as the chapter number) and `req.user` (the signed-in user).
2. Validate it. If something is wrong, show a message and stop.
3. Do the work with `db.get`, `db.all`, `db.run` or `db.transaction`.
4. Create notifications and an activity log entry.
5. Answer: `res.redirect(...)` after a form (the **Post/Redirect/Get** pattern, so refreshing does not send the form twice) or `res.render("page", data)` for a page.

The words `async` and `await` appear everywhere. Database calls take time, so they return a **promise**. `await` pauses that one handler until the answer arrives, while the server keeps serving other people.

7.2 routes/auth.js: accounts

- **Student registration** (`POST /register`) reads the name, email, student ID, password and ID photo, then checks them one by one. The first failure re-shows the form with a message, and the uploaded photo is deleted. Only when all checks pass is the user inserted and signed in.
- **Adviser registration** (`POST /register/adviser`) needs the invited email and the 6-digit code. The name, email and role come from the invitation, never from the form, so an adviser cannot choose their own role.
- **Forgot and reset password** were explained in Section 6.6.

routes/auth.js: creating the adviser account and closing the invitation together

```
  // The role, name and email come from the coordinator's invitation, not from this form.
  const userId = await db.transaction(async () => {
    const info = await db.run("INSERT INTO users (name, email, password_hash, role) VALUES (?, ?,
?, 'adviser')", [inv.name, inv.email, await bcrypt.hash(password, 10)]);
    await db.run("UPDATE adviser_invites SET status = 'Registered', used_at = NOW() WHERE id =
?", [inv.id]);
    return info.lastInsertRowid;
  });
```

Two writes happen in one transaction: the new user is inserted and the invitation is marked `Registered`. If one fails, neither is saved.

7.3 routes/student.js: the student's world

The first thing in this file is a guard that runs for every student route:

routes/student.js: finding the student's group

```
// Students without a group are sent to the create-group form.
router.use(async (req, res, next) => {
  req.group = await myGroup(req);
  const allowed = ['/', '/group'];
  if (!req.group && !allowed.includes(req.path)) return res.redirect('/student');
  next();
});
```

It loads the student's group into `req.group`. A student with no group can only reach the dashboard and the create-group form; everything else sends them back. Later handlers use `req.group`, which is why a student can never reach another group by editing the address.

routes/student.js: creating a group

```
router.post('/group', async (req, res) => {
  if (req.group) {
    req.flash('error', 'You are already in a group.');
    return res.redirect('/student');
  }
  const name = String(req.body.name || '').trim().slice(0, 255);
  const title = String(req.body.title || '').trim().slice(0, 1000);
  if (!name || !title) {
    return res.status(400).render('student/no-group', { title: 'Create your thesis group', form:
req.body, error: 'Enter a group name and a thesis title.' });
  }
  const emails = String(req.body.emails || '').split(/[\s,;]+/).map((e) =>
e.trim().toLowerCase()).filter(Boolean);
  const skipped = [];
  await db.transaction(async () => {
    const gid = (await db.run('INSERT INTO thesis_groups (name, title) VALUES (?, ?)', [name,
title])).lastInsertRowid;
    await db.run('INSERT INTO group_members (group_id, student_id) VALUES (?, ?)', [gid,
req.user.id]);
    for (const email of new Set(emails)) {
      const u = await db.get("SELECT id FROM users WHERE email = ? AND role = 'student' AND
active = 1", [email]);
      const taken = u && (await db.get('SELECT 1 FROM group_members WHERE student_id = ?',
[u.id]));
      if (!u || taken) { skipped.push(email); continue; }
      await db.run('INSERT INTO group_members (group_id, student_id) VALUES (?, ?)', [gid,
u.id]);
      await L.notify(u.id, `${req.user.name} added you to ${name}.`, '/student', 'student');
    }
    await L.createChapters(gid);
  });
  await L.log(req.user.id, `created the group ${name}`);
  req.flash(skipped.length ? 'warn' : 'ok', skipped.length
    ? `Group created. These emails were skipped because they are not free student accounts:
${skipped.join(', ')}.`
    : 'Group created. The coordinator will assign your adviser.');
  res.redirect('/student');
});
```

The group row, the members and the five chapters are created in one transaction. Emails that do not belong to free student accounts are skipped and reported in the message.

The upload route is the most important one in the system. The part after multer has accepted the file looks like this:

routes/student.js: saving a new version

```
    const v = await db.transaction(async () => {
      const v = ((await db.get('SELECT MAX(version_no) AS v FROM submissions WHERE chapter_id =
?', [c.id])).v || 0) + 1;
      await db.run('INSERT INTO submissions (chapter_id, version_no, original_name, stored_name,
uploaded_by) VALUES (?, ?, ?, ?, ?)',
        [c.id, v, req.file.originalname, req.file.filename, req.user.id]);
      await db.run("UPDATE chapters SET status = 'submitted' WHERE id = ?", [c.id]);
      return v;
    });
    await L.log(req.user.id, `submitted Chapter ${n} v${v} for ${req.group.name}`);
    if (req.group.adviser_id) await L.notify(req.group.adviser_id, `${req.group.name} submitted
Chapter ${n} (v${v}) for review.`, `/adviser/group/${req.group.id}/chapter/${n}`, 'student');
    req.flash('ok', `Chapter ${n} v${v} was sent to your adviser.`);
    res.redirect(back);
```

- A new `version_no` is the highest existing one plus one, so older versions are kept.
- Inside one transaction, the submission row is inserted and the chapter status becomes `submitted`.
- The adviser gets a notification with a link straight to the review page.

Before this part, the route also checked that the chapter exists, a file was chosen, the file looks like a real document, and the chapter is in a status that allows uploading (`none`, `revision` or `rejected`). An approved chapter is locked.

7.4 routes/adviser.js: the adviser's world

Every adviser route begins with `myGroup` (Section 6.7). The core is the review:

routes/adviser.js: a review decision

```
// Approve, return for revision or reject. A comment is required unless the chapter is approved.
router.post('/group/:id/chapter/:n/review', async (req, res) => {
  const g = await myGroup(req, req.params.id);
  if (!g) return denied(res);
  const c = await chapterOf(g.id, req.params.n);
  const back = `/adviser/group/${g.id}/chapter/${req.params.n}`;
  const decision = req.body.decision;
  const body = String(req.body.comment || '').trim().slice(0, 2000);
  if (!c || c.status !== 'submitted') { req.flash('error', 'Only a submitted chapter can be
reviewed.'); return res.redirect(back); }
  if (!['approved', 'revision', 'rejected'].includes(decision)) { req.flash('error', 'Choose a
decision.'); return res.redirect(back); }
  if (decision !== 'approved' && !body) { req.flash('error', 'Add a comment so the students know
what to fix.'); return res.redirect(back); }
  await db.transaction(async () => {
    if (body) await db.run('INSERT INTO comments (chapter_id, author_id, body) VALUES (?, ?, ?)',
[c.id, req.user.id, body]);
    await db.run('UPDATE chapters SET status = ? WHERE id = ?', [decision, c.id]);
  });
  const label = { approved: 'approved', revision: 'returned for revision', rejected: 'rejected'
}[decision];
  await L.log(req.user.id, `${label} ${g.name}, Chapter ${c.number}`);
  await L.notifyGroup(g.id, `Chapter ${c.number} was ${label} by ${req.user.name}.`,
`/student/chapter/${c.number}`, null, 'adviser');
  req.flash('ok', `Chapter ${c.number} marked ${L.STATUS[decision].label.toLowerCase()}.`);
  res.redirect(`/adviser/group/${g.id}`);
});
```

- Only a chapter in `submitted` status can be reviewed.

- The decision must be one of three allowed words, so the status column can never get an invalid value.
- A comment is required unless the chapter is approved.
- The comment and the new status are saved together in one transaction. Then the group is notified with the adviser as the sender.

Accepting or declining a group (`POST /assignments/:id`) works the same way: accepting updates the request and sets `adviser_id` on the group in one transaction; declining requires a reason, which is sent to the coordinator.

7.5 routes/admin.js: the coordinator

- **Users**: add students and panel members, activate or deactivate accounts. A forgotten password is replaced by a random temporary password built with `crypto.randomInt` (a secure random source), shown once to the coordinator.
- **Advisers**: invitations with a 6-digit code (Section 6.6), resend and cancel.
- **Panel members**: records with a random unusable password, because they never sign in.
- **Announcements**: saved, and one notification is created for every active user inside a transaction.
- **Reports**: the group list as a CSV file with a byte-order mark so Excel reads the characters correctly.

The most interesting handler is the group form, which sets the adviser, panel and defense schedule:

routes/admin.js: choosing an adviser

```
  // An adviser is only a request until they accept it. The group keeps its current adviser (if
any) meanwhile.
  const pending = await L.pendingRequestOf(g.id);
  const effective = pending ? pending.adviser_id : g.adviser_id;
  const adviserChanged = adviserId !== effective;
  const newRequest = adviserChanged && adviserId && adviserId !== g.adviser_id;

  await db.transaction(async () => {
    await db.run('UPDATE thesis_groups SET defense_date = ?, defense_time = ?, room = ? WHERE id
= ?',
      [date || null, date ? time : null, date ? room : null, g.id]);
    await db.run('DELETE FROM group_panelists WHERE group_id = ?', [g.id]);
    for (const p of panel) await db.run('INSERT INTO group_panelists (group_id, panel_id) VALUES
(?, ?)', [g.id, p]);
    if (adviserChanged) {
      await db.run("UPDATE adviser_requests SET status = 'Cancelled', decided_at = NOW() WHERE
group_id = ? AND status = 'Pending'", [g.id]);
      if (!adviserId) await db.run('UPDATE thesis_groups SET adviser_id = NULL WHERE id = ?',
[g.id]);
      if (newRequest) await db.run('INSERT INTO adviser_requests (group_id, adviser_id) VALUES
(?, ?)', [g.id, adviserId]);
    }
  });
```

Choosing an adviser does **not** assign them. It cancels any older pending request and creates a new `Pending` request. The group keeps its current adviser until the new one accepts. Scheduling a defense is validated first: valid date, not in the past, a listed time slot, a room, and at least two panel members.

The activity log page builds its filter dynamically but safely: the text of the SQL changes with the filters, while every value still travels in the placeholder list.

routes/admin.js: searching the activity log

```
const auditQuery = (q) => {
  const where = [];
  const args = [];
  if (q.q) { where.push('(a.action LIKE ? OR u.name LIKE ?)'); args.push(`%${q.q}%`, `%${q.q}%`);
}
  if (L.isDate(q.from)) { where.push('a.created_at >= ?'); args.push(q.from + ' 00:00:00'); }
  if (L.isDate(q.to)) { where.push('a.created_at <= ?'); args.push(q.to + ' 23:59:59'); }
  return { sql: where.length ? 'WHERE ' + where.join(' AND ') : '', args };
};
```

7.6 routes/shared.js: pages everyone uses

routes/shared.js: downloading a chapter file

```
// Download one uploaded version. Only people connected to that group can get it.
router.get('/files/:id', requireLogin, async (req, res) => {
  const sub = await db.get('SELECT s.*, c.group_id FROM submissions s JOIN chapters c ON c.id =
s.chapter_id WHERE s.id = ?', [Number(req.params.id)]);
  if (!sub || !(await L.canAccessGroup(req.user, sub.group_id))) {
    return res.status(403).render('error', { title: 'No access', message: 'You cannot open this
file.' });
  }
  const file = path.join(db.UPLOAD_DIR, path.basename(sub.stored_name));
  if (!fs.existsSync(file)) {
    return res.status(404).render('error', { title: 'File missing', message: 'The file is no
longer on the server.' });
  }
  res.download(file, sub.original_name);
});
```

The file is found by its database id, not by name. `canAccessGroup` decides whether this user may have it. `path.basename` strips any folder parts, so the file can only come from the uploads folder. `res.download` sends it under its original name.

The same file holds the Notifications page (read, mark as read, delete one, clear all) and My account (change your own password after checking the current one).

8. Pages and Styling: views and public

8.1 EJS in five minutes

Tag	Meaning
<%= value %>	Print the value with HTML **escaped**. A name like `<script>` appears as text instead of running. This is what protects against XSS (script injection).
<%- value %>	Print the value **raw**. Used only for trusted helper output such as the status badges.
<% code %>	Run JavaScript without printing (if, loops, variables).
<%- include("partials/header") %>	Insert another template here.

Data reaches a page through `res.render("page", { ... })` and through `res.locals` (Section 3.4).

8.2 The shared layout

Every page starts with `include("partials/header")` and ends with `include("partials/footer")`. The header draws the top bar, picks the menu for the user's role, shows the notification count and the flash message:

views/partials/header.ejs: a menu per role

```
<% const nav = {
  student: [['/student', 'My thesis']],
  adviser: [['/adviser', 'My groups']],
  admin: [['/admin', 'Overview'], ['/admin/users', 'Users'], ['/admin/groups', 'Groups'],
['/admin/advisers', 'Advisers'], ['/admin/panel', 'Panel members'], ['/admin/announcements',
'Announcements'], ['/admin/reports', 'Reports'], ['/admin/audit', 'Activity log']],
}; %>
```

The footer includes the chat widget for signed-in users, so the "Ask AI" button appears on every page.

8.3 A page that groups data: notifications.ejs

views/notifications.ejs: sorting messages into sections

```
<%
  const SECTIONS = [
    { key: 'adviser', title: 'From your adviser', mine: 'From advisers' },
    { key: 'coordinator', title: 'From the coordinator', mine: 'From the coordinator' },
    { key: 'student', title: 'From your group', mine: 'From students' },
    { key: 'system', title: 'Reminders', mine: 'Reminders' },
  ];
  const known = SECTIONS.map((s) => s.key);
  const groups = SECTIONS.map((s) => ({ ...s, rows: items.filter((n) => (known.includes(n.source)
? n.source : 'system') === s.key) })).filter((g) => g.rows.length);
%>
```

`SECTIONS` lists the four groups. `items.filter` picks the messages of each group, and an unknown or missing source falls back to "system". The page then loops over the groups that have messages and

prints a heading and a box for each message. Reading this file is a good first exercise because it uses variables, `map`, `filter` and loops in about 40 lines.

8.4 CSS

All styles are in `public/style.css`. The colours are defined once as **CSS variables** at the top (for example `--accent` for navy and `--gold`) and used everywhere. A `prefers-color-scheme: dark` block redefines them for dark mode, so the whole site switches theme without rewriting any rule.

On phones, tables turn into cards. The tables have `class="cards"`, and each cell has a `data-label` such as "Status". The CSS shows that label before the value:

public/style.css: one card per row on small screens

```
/* Phones: chapter and group tables become one card per row, so nothing scrolls sideways. */
@media (max-width:640px){
  table.cards thead{display:none}
  table.cards,table.cards tbody,table.cards tr,table.cards td{display:block;width:100%}
  table.cards tr{border:1px solid var(--line);border-radius:10px;padding:10px 12px;margin-
bottom:10px;background:var(--surface)}
  table.cards td{display:flex;justify-content:space-between;align-
items:center;gap:12px;border:0;padding:4px 0;text-align:right}
  table.cards td::before{content:attr(data-label);flex:0 0 auto;font-size:.72rem;letter-
spacing:.06em;text-transform:uppercase;color:var(--muted);font-weight:600;text-align:left}
  table.cards td:not([data-label])::before,table.cards td[data-label=""]::before{display:none}
  table.cards td:first-child{justify-content:flex-start;text-align:left;font-weight:600;padding-
bottom:6px}
  table.cards td:last-child:not([data-label]){justify-content:flex-start}
  table.cards .mono{overflow-wrap:anywhere;word-break:break-all}
  table.cards form.row{justify-content:flex-end}
}
```

8.5 Small scripts in the browser

Most of the work happens on the server, but a few pages contain a little browser JavaScript: the chat widget, the camera for the ID photo (`navigator.mediaDevices.getUserMedia`), and the spoken role choice on the sign-in page (`SpeechRecognition`). The chat widget sends the question without reloading the page:

views/partials/chatbot.ejs: asking the chatbot

```
    var body = new URLSearchParams({ _csrf: '<%= csrf %>', message: msg, history:
JSON.stringify(history) });
    fetch('/ai/chat', { method: 'POST', body: body })
      .then(function (r) { return r.json(); })
      .then(function (d) {
        if (d.error) { wait.textContent = d.error; return; }
        wait.textContent = d.reply; history.push({ role: 'user', content: msg }, { role:
'assistant', content: d.reply }); history = history.slice(-10);
      })
      .catch(function () { wait.textContent = 'Could not reach the server. Try again.'; });
```

Notice `URLSearchParams`: the server only reads form-style data (`express.urlencoded`), so the question is sent in that format together with the CSRF token. The answer is put into the page with `textContent`, never as HTML, so a reply can never inject code.

9. Notifications, Reminders and Email

9.1 Notifications

A notification is one row in `notifications` for one user. The routes create them with `L.notify` or `L.notifyGroup`, always with a link so clicking the message opens the right page. The header counts the unread ones (`is_read = 0`) on every request. Opening the Notifications page marks them read.

9.2 Deadline reminders: reminders.js

reminders.js: one reminder per chapter and deadline

```
  for (const c of rows) {
    const overdue = c.deadline < today;
    const kind = overdue ? 'overdue' : 'soon';
    const r = await db.run('INSERT IGNORE INTO deadline_reminders (chapter_id, kind, deadline)
VALUES (?, ?, ?)', [c.id, kind, String(c.deadline).slice(0, 10)]);
    if (!r.changes) continue;
    sent++;
    const link = `/student/chapter/${c.number}`;
    if (overdue) {
      await L.notifyGroup(c.gid, `Chapter ${c.number} was due on ${L.fmtDate(c.deadline)} and is
overdue.`, link);
      if (c.adviser_id) await L.notify(c.adviser_id, `${c.name}: Chapter ${c.number} is overdue
(due ${L.fmtDate(c.deadline)}).`, `/adviser/group/${c.gid}`);
    } else {
      await L.notifyGroup(c.gid, `Chapter ${c.number} is due on ${L.fmtDate(c.deadline)}.`,
link);
    }
  }
```

Every hour (and at start-up) the code finds chapters due within 3 days or already late whose students still owe work. For each one it tries `INSERT IGNORE INTO deadline_reminders`. That table has a unique key on (chapter, kind, deadline). If the reminder was already sent, the insert is ignored, `changes` is 0, and the loop skips it. This is a neat trick: the database itself prevents duplicates, so restarting the server never spams anyone. If the adviser changes the deadline, it counts as a new deadline and reminders can start again.

reminders.js: the hourly timer

```
function startReminders() {
  const run = () => sendReminders().then((n) => n && console.log(`Deadline reminders sent:
${n}.`)).catch((e) => console.error('Reminder check failed:', e.message));
  run();
  setInterval(run, 60 * 60 * 1000).unref();
}
```

`setInterval` repeats the check. `.unref()` tells Node that this timer alone should not keep the program alive.

9.3 Email: mailer.js

mailer.js: real email or a log file

```
async function send({ to, subject, text }) {
  if (configured()) {
    const port = Number(process.env.SMTP_PORT || 465);
    transport ||= nodemailer.createTransport({
      host: process.env.SMTP_HOST,
      port,
      secure: port === 465,
      auth: { user: process.env.SMTP_USER, pass: process.env.SMTP_PASS },
      // Give up quickly when there is no internet, so the page does not hang during a demo.
      connectionTimeout: 8000,
      greetingTimeout: 8000,
      socketTimeout: 12000,
    });
    await transport.sendMail({ from: process.env.MAIL_FROM || process.env.SMTP_USER, to, subject,
text });
    return { sent: true };
  }
  fs.mkdirSync(OUTBOX, { recursive: true });
  fs.appendFileSync(path.join(OUTBOX, 'mail.log'), `--- ${new Date().toISOString()}\nTo:
${to}\nSubject: ${subject}\n\n${text}\n\n`);
  console.log(`Email is not set up. The message to ${to} was saved in outbox/mail.log.`);
  return { sent: false };
}
```

If `SMTP_HOST`, `SMTP_USER` and `SMTP_PASS` are set in `.env`, the message is sent through Gmail (with an App Password, not your normal password). If not, the message is appended to `outbox/mail.log`. This is why the invitation code can always be recovered during a demo even without internet. The short timeouts stop the page from freezing when the Wi-Fi is bad.

9.4 First-run data: seed.js and bootstrap.js

On your own computer, an empty database is filled by `seedIfEmpty` with sample users (all with the password `password123`), four groups and sample chapters. On a live site (`NODE_ENV=production`) there is no sample data; `ensureAdmin` creates the first coordinator from the `ADMIN_*` settings and asks you to remove the password afterwards.

> **Watch out** `npm run seed` **wipes** the whole database before filling it. Back up first (Section 12).

10. The AI Modules

Four files add the AI features. They are small and independent, so you can study each one separately.

File	Job
ai.js	Talks to the free AI service (Gemini, or Ollama on your own computer) and hides the differences.
textextract.js	Reads the text out of PDF, DOCX and TXT files and remembers it.
plagiarism.js	Compares a chapter with other groups' uploads. Uses no AI service.
routes/ai.js	The "AI check" page, the grammar route, the chat route and the hourly limits.

10.1 ai.js: one door to the AI

ai.js: which AI is switched on

```
function provider() {
  if (process.env.GEMINI_API_KEY) return 'gemini';
  if (process.env.OLLAMA_MODEL) return 'ollama';
  return null;
}
```

Gemini is used when `GEMINI_API_KEY` is set, otherwise Ollama when `OLLAMA_MODEL` is set, otherwise the AI is "off" and the pages say so. The rest of the code just calls `ai.chat(...)` and does not care which one answers.

ai.js: calling the AI safely

```
async function post(url, body, headers = {}, attempt = 0) {
  let res;
  try {
    res = await fetch(url, {
      method: 'POST',
      headers: { 'content-type': 'application/json', ...headers },
      body: JSON.stringify(body),
      signal: AbortSignal.timeout(TIMEOUT_MS),
    });
  } catch (e) {
    throw new AiError(e.name === 'TimeoutError' ? 'The AI took too long to answer. Try again.' :
'Could not reach the AI service. Check the internet connection (or that Ollama is running).');
  }
  // 503 means the free model is busy for a moment, so wait briefly and try again (twice at
most).
  if (res.status === 503 && attempt < 2) {
    await new Promise((r) => setTimeout(r, 3000 * (attempt + 1)));
    return post(url, body, headers, attempt + 1);
  }
  if (res.status === 503) throw new AiError('The free AI is very busy right now. Try again in a
minute.');
  if (res.status === 429) throw new AiError('The free AI limit was reached for now. Wait a minute
(or until tomorrow) and try again.');
  if (!res.ok) {
    console.error('AI error', res.status, (await res.text()).slice(0, 300));
    throw new AiError(res.status === 400 || res.status === 403 ? 'The AI key or model setting was
refused. Check GEMINI_API_KEY and GEMINI_MODEL in .env.' : 'The AI service had a problem. Try
again later.');
  }
  return res.json();
}
```

- `fetch` sends the request. `AbortSignal.timeout` gives up after 60 seconds.
- A 503 ("busy") is retried twice after a pause; after that the user gets a clear message.
- A 429 means the free limit is reached, and 400 or 403 means the key or model name is wrong.
- Errors are turned into an `AiError` with a friendly message, so the routes can show it without exposing technical details. The key itself travels in a header, never in the address.

10.2 textextract.js: reading files

textextract.js: read once, remember

```
async function textOf(sub) {
  const cached = await db.get('SELECT body FROM submission_text WHERE submission_id = ?',
[sub.id]);
  if (cached) return cached.body;
  let text;
  try {
    text = await readFile(path.join(db.UPLOAD_DIR, path.basename(sub.stored_name)),
sub.original_name);
  } catch (e) {
    if (e instanceof ExtractError) throw e;
    console.error('extract failed', sub.id, e.message);
    throw new ExtractError('That file could not be read. It may be scanned images, protected, or
damaged.');
  }
  text = text.replace(/^-- \d+ of \d+ --$/gm, '').replace(/\r/g, '').replace(/[ \t]+/g, '
').replace(/\n{3,}/g, '\n\n').trim();
  if (text.split(/\s+/).length < 30) throw new ExtractError('There is almost no text in this file
(it may be a scan). A checker needs selectable text.');
  await db.run('INSERT IGNORE INTO submission_text (submission_id, body) VALUES (?, ?)', [sub.id,
text]);
  return text;
}
```

`textOf` first looks in `submission_text`. If the text is not there, it reads the file (PDF with pdf-parse, DOCX with mammoth, TXT directly), tidies spaces and blank lines, refuses files with almost no text (scans), and saves the result. The second check of the same file is instant.

10.3 plagiarism.js: how the similarity score works

The idea is simple and explainable. Cut each document into overlapping runs of 6 words (called **shingles**). If two documents share many runs, they share wording.

plagiarism.js: turning text into 6-word runs

```
const N = 6; // a "match" is 6 words in a row that also appear in the other text

const words = (s) => s.toLowerCase().replace(/[^\p{L}\p{N}\s]/gu, '
').split(/\s+/).filter(Boolean);
function shingles(ws) {
  const out = new Set();
  for (let i = 0; i + N <= ws.length; i++) out.add(ws.slice(i, i + N).join(' '));
  return out;
}
```

A worked example

Take the sentence "the system sends warnings to residents by text message". It has 9 words, so it gives 9 - 6 + 1 = 4 runs: "the system sends warnings to residents", "system sends warnings to residents by", "sends warnings to residents by text" and "warnings to residents by text message". If another group's file contains the same sentence, all 4 runs match. A different sentence about the same topic usually shares none, because 6 words in the same order is rare by chance.

plagiarism.js: counting matches against other groups

```
  const matched = new Set();
  const sources = [];
  let skipped = 0;
  for (const o of others) {
    let theirs;
    try { theirs = shingles(words(await textOf(o))); } catch (e) { skipped++; continue; }
    let hits = 0;
    for (const s of mine) if (theirs.has(s)) { hits++; matched.add(s); }
    if (hits) sources.push({ group: o.group_name, chapter: o.chapter_no, percent:
Math.round((hits / mine.size) * 100) });
  }
  sources.sort((a, b) => b.percent - a.percent);
```

- `mine` is the set of runs in the chapter being checked. For every other group's latest upload, each run is looked up (`theirs.has(s)`).
- Matched runs are collected in one set, so a run found in three files is counted once.
- The score is `matched.size / mine.size`: the share of this chapter's runs that also appear elsewhere.
- Per-file percentages give the "closest files" list. Students see "Another group" instead of the name, but advisers and the coordinator see it.

plagiarism.js: finding the passages to show

```
  // Sentences of this chapter where most word-runs were found elsewhere.
  const sentences = [];
  for (const raw of (await textOf(sub)).split(/(?<=[.!?])\s+|\n+/)) {
    const sh = [...shingles(words(raw))];
    if (sh.length >= 3 && sh.filter((s) => matched.has(s)).length / sh.length >= 0.5)
sentences.push(raw.trim().slice(0, 300));
    if (sentences.length >= 8) break;
  }
```

A sentence is shown as a "matching passage" when at least half of its runs were matched (and it has at least 3 runs). The page labels the score Low below 10%, Check from 10%, and High from 25% (`views/ai-tools.ejs`).

> **Key idea** This checker only compares with work already inside ThesisTrack. It cannot see the internet, books or journals. That limit is stated on the page and in the system document, and it is the honest answer if a panelist asks.

10.4 routes/ai.js: the routes around the tools

routes/ai.js: a fair share for everyone

```
// The free AI services have daily limits, so each person gets a fair share per hour.
const used = new Map();
function allow(userId, kind, max) {
  const key = `${userId}:${kind}`;
  const now = Date.now();
  const recent = (used.get(key) || []).filter((t) => now - t < 3600_000);
  if (recent.length >= max) { used.set(key, recent); return false; }
  recent.push(now);
  used.set(key, recent);
  return true;
}
```

`allow` keeps, for each person and tool, the times of recent requests in a `Map` (memory). If someone already used up their hourly allowance (20 similarity checks, 15 grammar checks, 40 chat messages), the request is refused. Because the map lives in memory, it resets when the server restarts. That is fine for a small system.

routes/ai.js: the same access rule as downloads

```
async function loadSubmission(req, res) {
  const sub = await db.get(
    `SELECT s.*, c.group_id, c.number AS chapter_no, g.name AS group_name FROM submissions s
       JOIN chapters c ON c.id = s.chapter_id JOIN thesis_groups g ON g.id = c.group_id WHERE
s.id = ?`,
    [Number(req.params.id)]
  );
  if (!sub || !(await L.canAccessGroup(req.user, sub.group_id))) {
    res.status(403).render('error', { title: 'No access', message: 'You cannot open this file.'
});
    return null;
  }
  return sub;
}
```

Every AI check starts by loading the submission and asking `canAccessGroup`. A student of another group gets a 403 page.

routes/ai.js: grammar suggestions

```
router.post('/submission/:id/grammar', async (req, res) => {
  const sub = await loadSubmission(req, res);
  if (!sub) return;
  if (!allow(req.user.id, 'grammar', 15)) return page(res, sub, { error: 'You have used many
grammar checks this hour. Try again later.' });
  try {
    const all = (await textOf(sub)).split(/\s+/);
    const parts = Math.ceil(all.length / PART_WORDS);
    const part = Math.min(Math.max(Number(req.body.part) || 1, 1), parts);
    const text = all.slice((part - 1) * PART_WORDS, part * PART_WORDS).join(' ');
    const raw = await ai.chat({ system: GRAMMAR_SYSTEM, messages: [{ role: 'user', content: text
}], json: true, maxTokens: 2500 });
    let issues = [];
    try { issues = (JSON.parse(raw).issues || []).filter((i) => i && i.original &&
i.suggestion).slice(0, 15); }
    catch (e) { return page(res, sub, { error: 'The AI answer could not be read. Try again.' });
}
    page(res, sub, { grammar: { issues, part, parts } });
  } catch (e) {
    if (!(e instanceof ExtractError || e instanceof ai.AiError)) throw e;
    page(res, sub, { error: e.message });
  }
});
```

- The text is cut into parts of 1,200 words, and only the chosen part is sent.
- The prompt tells the AI to answer only in JSON, and the answer is read with `JSON.parse` inside a `try` block. If the AI answers something unreadable, the user sees a message instead of a crash.
- The first 15 valid fixes are shown. Each has the original text, the suggestion and the reason.

routes/ai.js: the chatbot endpoint

```
router.post('/chat', async (req, res) => {
  if (!ai.provider()) return res.json({ error: 'The assistant is not switched on yet. Ask the
coordinator to add a free AI key.' });
  if (!allow(req.user.id, 'chat', 40)) return res.json({ error: 'You have asked a lot this hour.
Please try again later.' });
  let history = [];
  try { history = JSON.parse(req.body.history || '[]'); } catch (e) { /* start fresh */ }
  const messages = (Array.isArray(history) ? history : [])
    .filter((m) => m && (m.role === 'user' || m.role === 'assistant') && typeof m.content ===
'string')
    .slice(-10).map((m) => ({ role: m.role, content: m.content.slice(0, 1500) }));
  const message = String(req.body.message || '').trim().slice(0, 1500);
  if (!message) return res.json({ error: 'Type a question first.' });
  messages.push({ role: 'user', content: message });
  while (messages[0].role !== 'user') messages.shift();
  try {
    res.json({ reply: await ai.chat({ system: await chatSystem(req.user), messages, maxTokens:
700 }) });
  } catch (e) {
    if (!(e instanceof ai.AiError)) throw e;
    res.json({ error: e.message });
  }
});
```

The chat route accepts only the last 10 messages, trims each to 1,500 characters, and builds a **system prompt**: instructions that tell the AI it is the ThesisTrack assistant, that it must not write the thesis for the students, and (for students) the status and deadlines of their own group. The question is then sent to `ai.chat` and the reply is returned as JSON to the widget.

> **Try it** Switch the AI on and off by adding or removing `GEMINI_API_KEY` in `.env` and restarting. Watch the "AI check" page and the "Ask AI" button change their messages. The similarity check keeps working either way.

11. Following a Feature from Click to Database

The best way to understand a system is to follow one action all the way. Three examples follow. Open the named files and find each step.

11.1 A student uploads Chapter 4

1. The student opens `/student/chapter/4`. `routes/student.js` (`router.get("/chapter/:n")`) loads the chapter, its versions and comments and renders `views/student/chapter.ejs`.
2. The student picks a file and presses Upload. The browser sends a **multipart** POST to `/student/chapter/4/upload` with the CSRF token inside the form.
3. `server.js`: security headers, session, per-request setup. The global CSRF check skips multipart forms on purpose (Section 3.5).
4. `requireRole("student")` confirms the role. The guard in `routes/student.js` loads `req.group`.
5. multer saves the file under a random name and enforces the type and the 10 MB limit.
6. `csrfCheck` now verifies the token. The route checks that the chapter exists, a file was sent, its first bytes look like a PDF or Word file, and the chapter's status allows an upload.
7. One transaction inserts a row in `submissions` (next version number) and sets `chapters.status` to `submitted`.
8. `L.log` writes to `activity_log`. `L.notify` writes a notification for the adviser with source "student".
9. The route flashes "sent to your adviser" and redirects to the chapter page. The adviser's header now shows one more unread notification.

11.2 The adviser returns it for revision

1. The adviser opens the notification link to `/adviser/group/:id/chapter/4`. `myGroup` proves the group is theirs and `views/adviser/review.ejs` shows versions and comments.
2. They write a comment and press "Return for revision". The form posts to `/adviser/group/:id/chapter/4/review`.
3. The route checks the status is `submitted`, the decision is valid and a comment exists.
4. One transaction inserts the comment and sets the status to `revision`.
5. `L.notifyGroup` creates a notification for each student with source "adviser". On the students' Notifications page it appears under "From your adviser".
6. The chapter page now allows an upload again, because `canUpload` is true for `revision`. The cycle repeats until approval.

11.3 A student runs the similarity check

1. On the chapter page, the student clicks "AI check" next to a version. The browser requests `/ai/submission/:id`.
2. `routes/ai.js` loads the submission and asks `canAccessGroup`. A student of another group would get 403.

3. `views/ai-tools.ejs` shows the two tools. The student presses "Check similarity", a POST to `/ai/submission/:id/plagiarism`.
4. `allow` confirms the hourly limit. `plagiarism.check` runs.
5. `textOf` reads the chapter (from `submission_text` if saved, otherwise from the file) and then each other group's latest file.
6. Runs of 6 words are compared, the percentage and the matching passages are computed, and the page is shown. An entry is written to the activity log.

12. Running, Testing and Backing Up

12.1 The npm commands

Command	What it does
npm start	Starts the server (`node server.js`). Start MySQL in XAMPP first.
npm run dev	Starts the server and restarts it when a file changes. Handy while coding.
npm run seed	WIPES the database and fills it with sample data.
npm test	Runs `scripts/smoke-test.js` against a running server.
npm run backup	Saves the database (with mysqldump) and the uploads folder into `backups/<date>`.

Double-clicking `start-thesistrack.bat` runs `npm start` and opens the browser.

12.2 The smoke test

`scripts/smoke-test.js` is a robot user. It talks to the running server over HTTP, signs in as the sample users, uploads files, reviews chapters, books consultations, resets passwords and checks that the right things happen and the wrong things are refused (for example, "students cannot open the adviser pages"). It prints PASS or FAIL for each check (about 100). It was written to run against a **fresh** sample database: run `npm run seed`, then `npm start` in one terminal and `npm test` in another. Running it twice in a row will show failures because the first run changed the data.

12.3 The settings (.env)

Setting	Meaning
SESSION_SECRET	Signs the login cookie. Required on the live site.
NODE_ENV=production	Switches to live mode.
PORT	Port to listen on (default 3000).
DB_HOST, DB_PORT, DB_USER, DB_PASSWORD, DB_NAME	MySQL connection (the defaults fit XAMPP).
UPLOAD_DIR	Where uploaded files are kept (default: the uploads folder).
SMTP_HOST, SMTP_PORT, SMTP_USER, SMTP_PASS, MAIL_FROM	Email through Gmail (use an App Password).
APP_URL	The public address put in password reset links.
ADMIN_EMAIL, ADMIN_PASSWORD, ADMIN_NAME	The first coordinator on a live site.

Setting	Meaning
GEMINI_API_KEY, GEMINI_MODEL	The free Google AI (see `AI-SETUP.md`).
OLLAMA_MODEL, OLLAMA_URL	The alternative AI that runs on your own computer.

12.4 When something goes wrong

- Read the **terminal window** where the server runs. Errors are printed there (`console.error` in the error handler).
- "The database is not reachable": start MySQL in the XAMPP Control Panel.
- "Your form expired": the page was open too long or the session ended. Refresh and try again.
- To see what a variable holds, temporarily add `console.log(variable)` in the route, reload the page and look at the terminal.

13. Where to Change Things

I want to change...	Look here	Setting
The names of the five chapters	lib.js	CH_NAMES
The consultation time slots	lib.js	SLOTS
Colours and dark mode	public/style.css	:root variables
The menu of a role	views/partials/header.ejs	nav object
Upload size and allowed types for chapters	routes/student.js	ALLOWED, fileSize
Minimum password length	routes/auth.js, routes/shared.js, routes/admin.js	password.length < 8
Failed-login lockout	routes/auth.js	10 attempts, 15 minutes
Password reset link lifetime	routes/auth.js	RESET_MINUTES
Adviser invitation code lifetime	routes/admin.js	OTP_MINUTES
How long a login lasts on the server	server.js	SESSION_MS
Days before a deadline to remind	reminders.js	SOON_DAYS
Notification sections and labels	views/notifications.ejs	SECTIONS
AI hourly limits	routes/ai.js	allow(...) numbers
AI model	.env	GEMINI_MODEL
Similarity word-run length	plagiarism.js	N
Low, Check and High thresholds	views/ai-tools.ejs	10 and 25
The chatbot's instructions	routes/ai.js	chatSystem
Number of panel members needed for a defense	routes/admin.js	panel.length < 2

Practice exercises

Make each change on a copy of the project or on your sample data, restart the server and see the result.

1. Add a new consultation time slot, `16:00`. Which file? Which pages show it?
2. Change the reminder window from 3 days to 5 days. How can you see the effect?
3. Make the minimum password 10 characters. How many places must change? (Hint: search the project for `length < 8`.)
4. Change the notification section title "From your group" to "From classmates".
5. Add a fifth colour variable to the CSS and use it for something small.
6. Read `views/student/dashboard.ejs` and list every piece of data it receives from `routes/student.js`.

14. Glossary

Term	Meaning
Server / client	The program that answers requests / the browser that asks.
HTTP request and response	The question the browser sends and the answer the server returns.
Route	A function connected to an address and a method such as GET or POST.
GET and POST	GET asks for a page. POST sends data, such as a form.
Middleware	Code that runs before a route and can stop or allow the request.
Session and cookie	A cookie is a small value the browser keeps; the session is the data the server keeps for that cookie.
Template (EJS)	HTML with places where data is filled in.
Partial	A template piece shared by many pages, such as the header.
Flash message	A message shown once on the next page, such as "Saved."
Hash	A one-way scramble of a value. Used for passwords and codes.
CSRF	An attack that makes your browser send a hidden form. Stopped by a secret token.
XSS	An attack that puts a script into a page. Stopped by escaping text.
SQL injection	An attack that puts SQL into an input. Stopped by placeholders.
Transaction	A group of database changes that all succeed or all cancel.
Foreign key	A column that stores the id of a row in another table.
Primary key	The column that identifies each row.
Environment variable	A setting kept outside the code, read with process.env.
Promise, async and await	A way to wait for slow work (such as the database) without freezing the server.
Connection pool	A set of ready database connections that requests share.
Magic bytes	The first bytes of a file, which reveal its real type.
Rate limit	A cap on how often something may be done.
Shingle	A short run of consecutive words used to compare documents.
System prompt	The hidden instructions that tell an AI how to behave.

15. Review Questions

Use these to test yourselves before the defense. Each answer points to the section that explains it.

> **Remember** You do not need to memorise the code. You need to understand the **path of a request**, the **security layers**, and the **reason** for each design choice. If you can explain those, you can answer any code question the panel asks.

Question	Short answer
Why does the server use a database for sessions instead of memory?	People stay signed in after a restart, and several copies of the app could share sessions (Sections 3.3 and 6.4).
How does the system stop one group from reading another group's chapters?	Routes work from the logged-in user's own group, adviser routes filter by adviser_id, and files and AI checks go through canAccessGroup (6.7, 5.6).
What is CSRF and how is it prevented?	A forged form sent by another site while you are signed in. Every form carries a secret token that is checked (3.5).
Why are passwords hashed and not encrypted?	A hash cannot be reversed, so even a stolen database does not reveal passwords (6.2).
Why use a transaction when creating a group?	The group, members and five chapters must all be created or none, to avoid a half-built group (4.6, 7.3).
What stops SQL injection?	Placeholders: values are sent separately from the SQL text (4.5).
Why does a wrong email and a wrong password give the same message?	So nobody can find out which emails are registered (6.3).
How can a renamed .exe be rejected as a PDF?	The first bytes are checked, not just the file name (6.5).
Why can a student not assign their own adviser?	Only the coordinator route changes the adviser, and the adviser must accept the request first (7.5).
How does the system avoid sending the same reminder twice?	A unique key in deadline_reminders with INSERT IGNORE (9.2).
How does the plagiarism check work, and what can it not do?	It compares overlapping 6-word runs with other groups' uploads. It cannot search the internet or read scanned files (10.3).
Why does the similarity check not use AI?	It does not need to, it stays private (nothing leaves the server) and it costs nothing (10.3).
What happens if the free AI is busy or the key is wrong?	ai.js retries a busy answer, and turns errors into friendly messages (10.1).
Why is the chatbot reply inserted with textContent?	So a reply can never run as code in the page (8.5).

Question	Short answer
How do the notification sections work?	Each notification stores a source; the page groups them by it (5.7, 8.3).
Why can panel members not sign in?	The advisers recommended keeping them as records, so the login route refuses that role (6.3).